

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['figure.facecolor'] = 'white'
plt.rcParams['axes.facecolor'] = 'white'
sns.set_style('whitegrid')

df =
pd.read_csv('/home/claude/healthcare_lifestyle_data
.csv')

report = []
report.append("# EDA Report: Patient Lifestyle &
Disease Risk Dataset\n")
report.append(f"**Dataset shape:** {df.shape[0]}
rows, {df.shape[1]} columns\n")

# ----- 1. DATA QUALITY / FORENSIC CHECK -----
-----
report.append("## 1. Data Quality Audit\n")

n_dupes = df.duplicated().sum()
df_clean = df.drop_duplicates().copy()

missing = df_clean.isna().sum()
missing_pct = (missing / len(df_clean) *
100).round(1)
missing_table = pd.DataFrame({'missing_count':
missing, 'missing_pct': missing_pct})
missing_table =
missing_table[missing_table['missing_count'] > 0]

total_cells = df_clean.shape[0] * df_clean.shape[1]
total_missing = df_clean.isna().sum().sum()

```

```

completeness = round((1 - total_missing /
total_cells) * 100, 1)

report.append(f"- Duplicate rows found and removed:
**{n_dupes}**")
report.append(f"- Overall data completeness after
dedup: **{completeness}%**")
report.append(f"- Columns with missing values:\n")
report.append(missing_table.to_markdown())
report.append("")

# ----- 2. UNIVARIATE / DESCRIPTIVE STATS -----
-----
report.append("## 2. Descriptive Statistics (Five-
Number Summary)\n")
numeric_cols = ['age', 'bmi', 'sleep_hours',
'exercise_hours_per_week',
'          'alcohol_units_per_week',
'cholesterol', 'bp_systolic',
'          'glucose_level',
'disease_risk_score']
desc = df_clean[numeric_cols].describe().T.round(2)
report.append(desc.to_markdown())
report.append("")

# Mean vs Median skew check
report.append("### Mean vs. Median – flagging
skewed variables\n")
skew_notes = []
for col in numeric_cols:
    mean_v = df_clean[col].mean()
    median_v = df_clean[col].median()
    pct_diff = abs(mean_v - median_v) / median_v *
100 if median_v != 0 else 0
    if pct_diff > 5:
        direction = "right-skewed (mean > median)"
    if mean_v > median_v else "left-skewed (mean <
median)"
    skew_notes.append(f"- **{col}**: mean=
{mean_v:.1f}, median={median_v:.1f} →

```

```

{direction}")
report.append("\n".join(skew_notes) if skew_notes
else "No strongly skewed variables detected.")
report.append("")

# ----- 3. OUTLIER DETECTION (IQR method) -----
report.append("## 3. Outlier Detection (IQR
Method)\n")
outlier_summary = []
for col in numeric_cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df_clean[(df_clean[col] < lower) |
(df_clean[col] > upper)]
    outlier_summary.append({
        'column': col, 'lower_bound': round(lower,
1), 'upper_bound': round(upper, 1),
        'n_outliers': len(outliers)
    })
outlier_df = pd.DataFrame(outlier_summary)
report.append(outlier_df.to_markdown(index=False))
report.append("")
report.append("***Notable flagged records:***")
chol_outliers = df_clean[df_clean['cholesterol'] >
400][['patient_id', 'cholesterol']]
bp_outliers = df_clean[df_clean['bp_systolic'] >
190][['patient_id', 'bp_systolic']]
bmi_outliers = df_clean[(df_clean['bmi'] > 55) |
(df_clean['bmi'] < 10)][['patient_id', 'bmi']]
report.append(f"- Cholesterol > 400 (likely entry
error, e.g. 520 instead of 52.0 or misread digit):
{chol_outliers.to_dict('records')}")
report.append(f"- BP systolic > 190 (likely genuine
hypertensive crisis cases, needs clinical review
not deletion): {bp_outliers.to_dict('records')}")
report.append(f"- BMI outliers (mix of entry error

```

```

and genuine extreme):
{bmi_outliers.to_dict('records')}}")
report.append("")

# ----- 4. CORRELATION -----
report.append("## 4. Correlation Analysis\n")
corr =
df_clean[numeric_cols].corr(numeric_only=True).round(2)
report.append(corr.to_markdown())
report.append("")
top_corr_with_risk =
corr['disease_risk_score'].drop('disease_risk_score').sort_values(key=abs, ascending=False)
report.append("***Strongest correlations with disease_risk_score***")
for col, val in top_corr_with_risk.items():
    report.append(f"- {col}: r = {val}")
report.append("")
report.append("*Note: correlation is a clue, not a verdict – cholesterol, BP and BMI all rise with age, so age is a likely confounder behind several of these relationships, not proof of independent causation.*")
report.append("")

# ----- 5. GROUPED / CATEGORICAL BREAKDOWN -----
report.append("## 5. Risk by Smoking Status\n")
smoke_group = df_clean.groupby('smoking_status')['disease_risk_score'].agg(['mean', 'median', 'count']).round(1)
report.append(smoke_group.to_markdown())
report.append("")

# ----- SAVE CLEANED DATA -----
df_clean.to_csv('/home/claude/healthcare_lifestyle_data_cleaned.csv', index=False)

with open('/home/claude/eda_report.md', 'w') as f:

```

```
f.write("\n".join(report))
```

```
print("Report generated.")  
print(f"Duplicates removed: {n_dupes}")  
print(outlier_df)
```